STEP SEM-3 CODINCLUB • POWERED BY BRIDGELABZ

Category C Assignment Problems

EASY TIER

PROBLEM 1. Easy • Fantasy Team Score Multiplier

Topics: Creating/Modifying Arrays, Arrays Passed by Reference

Problem Statement:

In a fantasy sports app, every user picks a Captain (2x points) and a Vice-Captain (1.5x points) from their lineup. Write a method that applies both multipliers directly to the lineup's score array, so the app's scoreboard reflects the boosted totals immediately after the match end.

Requirements:

- `applyMultipliers(...)` must modify the caller's original array directly; it should return nothing at all.
- Only the captain's and vice-captain's positions change; every other score stays exactly as it was.

Function Signature:

```
static void applyMultipliers(double[] playerScores, int captainIndex, int
viceCaptainIndex)
```

Solution Code:

```
static void applyMultipliers(double[] playerScores, int captainIndex, int
viceCaptainIndex) {
    playerScores[captainIndex] *= 2.0;
    playerScores[viceCaptainIndex] *= 1.5;
}
```

EASY TIER (Continuation)

PROBLEM 2. Easy • Duplicate Player Pick Checker

Topics: Arrays, Strings, Nested Loops

Problem Statement:

A glitch in the fantasy app's draft screen occasionally lets a user tap the same player twice before the UI catches up, silently adding them to the lineup twice. Write a method that checks a submitted lineup for a repeated player name before it's accepted.

Requirements:

- Compare every name against every other name using plain nested loops—no Collections class of any kind.
- Report the first duplicate found, scanning in order; if none exist, say so clearly.

Function Signature:

```
static String findDuplicatePick(String[] playerNames)
```

Solution Code:

```
static String findDuplicatePick(String[] playerNames) {
    for (int i = 0; i < playerNames.length; i++) {
        for (int j = i + 1; j < playerNames.length; j++) {
            if (playerNames[i].equals(playerNames[j])) {
                return "Duplicate Found: " + playerNames[i];
            }
        }
    }
    return "No Duplicates Found";
}
```

INTERMEDIATE TIER

PROBLEM 3. Intermediate • Top Performer Tracker

Topics: Arrays, Loops, Logical Thinking

Problem Statement:

A fantasy league's weekly recap wants to instantly call out the week's standout performer and the week's biggest disappointment, along with how wide the gap between them was without sorting the entire scoreboard just to read off two numbers.

Requirements:

- Do not sort the array—find the minimum and maximum in a single pass through the array.
- Report the minimum, the maximum, and the spread (max - min) together.

Function Signature:

```
static String findMinMaxSpread(int[] scores)
```

Solution Code:

```
static String findMinMaxSpread(int[] scores) {
    int min = scores[0];
    int max = scores[0];
    for (int i = 1; i < scores.length; i++) {
        if (scores[i] < min) {
            min = scores[i];
        }
        if (scores[i] > max) {
            max = scores[i];
        }
    }
    int spread = max - min;
    return "Min: " + min + "\nMax: " + max + " Spread: " + spread;
}
```

INTERMEDIATE TIER (Continuation)

PROBLEM 4. Intermediate • Match Day Grid Analyzer

Topics: 2D Arrays, User-Defined Methods, Loops

Problem Statement:

A cricket stats app logs runs scored in every over of every match as a grid—one row per match, one column per over. The app wants to flag which matches were genuine "Power Surge" innings without repeating averaging logic.

Requirements:

- Write a private helper, `rowAverage(int[] row)`, and call it once per match from the main method.
- A match averaging at or above threshold is a "Power Surge"; below it, "Normal".

Function Signatures:

```
static double rowAverage(int[] row)
static String classifyMatches(int[][] runsPerOver, int threshold)
```

Solution Code:

```
static double rowAverage(int[] row) {
    if (row.length == 0) return 0.0;
    double sum = 0;
    for (int val : row) { sum += val; }
    return sum / row.length;
}

static String classifyMatches(int[][] runsPerOver, int threshold) {
    StringBuilder sb = new StringBuilder();
    for (int i = 0; i < runsPerOver.length; i++) {
        double avg = rowAverage(runsPerOver[i]);
        sb.append("Match ").append(i).append(": ")
          .append(avg >= threshold ? "Power Surge" : "Normal");
        if (i < runsPerOver.length - 1) sb.append(" | ");
    }
    return sb.toString();
}
```

ADVANCED TIER

PROBLEM 5. Advanced • Fantasy League Auto-Draft Ranking Engine

Topics: Method Overloading, Comparable, Arrays.sort

Problem Statement:

An auto-draft feature needs to decide which players are draftable and rank them by fantasy points descending using standard library sorting.

Function Signatures & Solution:

```
class Player implements Comparable<Player> {
    String name; int matchesPlayed; double battingAverage; boolean injured;
    public Player(String name, int mp, double ba, boolean inj) {
        this.name = name; this.matchesPlayed = mp;
        this.battingAverage = ba; this.injured = inj;
    }
    public int compareTo(Player other) {
        return Double.compare(other.battingAverage, this.battingAverage);
    }
}
static boolean isDraftable(int mp) { return mp >= 10; }
static boolean isDraftable(int mp, boolean inj) { return mp >= 5 && !inj; }
static String draftAndRank(Player[] players) {
    int count = 0;
    for (Player p : players) if (p != null && (isDraftable(p.matchesPlayed) ||
isDraftable(p.matchesPlayed, p.injured))) count++;
    Player[] draftable = new Player[count];
    int idx = 0;
    for (Player p : players) if (p != null && (isDraftable(p.matchesPlayed) ||
isDraftable(p.matchesPlayed, p.injured))) draftable[idx++] = p;
    java.util.Arrays.sort(draftable);
    StringBuilder sb = new StringBuilder();
    for (int i = 0; i < draftable.length; i++) {
        sb.append((i + 1)).append(". ").append(draftable[i].name);
        if (i < draftable.length - 1) sb.append(" ");
    }
    return sb.toString();
}
```